

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	6
1.1 Описание входных данных.....	8
1.2 Описание выходных данных.....	9
2 МЕТОД РЕШЕНИЯ.....	12
3 ОПИСАНИЕ АЛГОРИТМОВ.....	14
3.1 Алгоритм метода set_head класса cl_base.....	14
3.2 Алгоритм метода delete_subobject класса cl_base.....	16
3.3 Алгоритм метода get_subobject_by_coordinate класса cl_base.....	16
3.4 Алгоритм метода build_tree_objects класса cl_application.....	18
3.5 Алгоритм метода exes_app класса cl_application.....	20
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	25
5 КОД ПРОГРАММЫ.....	41
5.1 Файл cl_2.cpp.....	41
5.2 Файл cl_2.h.....	41
5.3 Файл cl_3.cpp.....	41
5.4 Файл cl_3.h.....	42
5.5 Файл cl_4.cpp.....	42
5.6 Файл cl_4.h.....	43
5.7 Файл cl_5.cpp.....	43
5.8 Файл cl_5.h.....	43
5.9 Файл cl_6.cpp.....	44
5.10 Файл cl_6.h.....	44
5.11 Файл cl_application.cpp.....	44
5.12 Файл cl_application.h.....	49
5.13 Файл cl_base.cpp.....	50
5.14 Файл cl_base.h.....	58

5.15 Файл main.cpp.....	59
6 ТЕСТИРОВАНИЕ.....	61
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	64

1 ПОСТАНОВКА ЗАДАЧИ

Иметь возможность доступа из текущего объекта к любому объекту системы, «мечта» разработчика программы.

Расширить функциональность базового класса:

- метод переопределения головного объекта для текущего в дереве иерархии. Метод должен иметь один параметр, указатель на объект базового класса, содержащий указатель на новый головной объект. Переопределение головного объект для корневого объекта недопустимо. Недопустимо создать второй корневой объект. Недопустимо при переопределении, чтобы у нового головного появились два подчиненных объекта с одинаковым наименованием. Новый головной объект не должен принадлежать к объектам из ветки текущего. Если переопределение выполнено, метод возвращает значение «истина», иначе «ложь»;

- метод удаления подчиненного объекта по наименованию. Если объект не найден, то метод завершает работу. Один параметр строкового типа, содержит наименование удаляемого подчиненного объекта;

- метод получения указателя на любой объект в составе дерева иерархии объектов согласно пути (координаты). В качестве параметра методу передать путь (координату) объекта. Координата задаться в следующем виде:

- о / - корневой объект;
- о //«имя объекта» - поиск объекта по уникальной имени от корневого (для однозначности уникальность требуется в рамках дерева);
- о . - текущий объект;
- о .«имя объекта» - поиск объекта по уникальной имени от

текущего (для однозначности уникальность требуется в рамках ветви дерева от текущего объекта);

о «имя объекта 1»[/«имя объекта 2»] . . . - относительная координата от текущего объекта, «имя объекта 1» подчиненный текущего;

о /«имя объекта 1»[/«имя объекта 2»] . . . - абсолютная координата от корневого объекта.

Примеры координат:

```
/
//ob_3
.
.ob_2
ob_2/ob_3
/ob_1/ob_2/ob_3
```

Если координата - пустая строка или объект не найден или определяется неоднозначно (дуближ имен на ветке, на дереве), тогда вернуть нулевой указатель.

Наименование объекта не содержит символы «.» и «/».

Система содержит объекты пяти классов, не считая корневого. Номера классов: 2,3,4,5,6.

Состав и иерархия объектов строиться посредством ввода исходных данных. Ввод организован как в версии № 2 курсовой работы. Единственное различие. В строке ввода первым указано не наименование головного объекта, а абсолютный путь к нему. При построении дерева уникальность наименования относительно множества непосредственно подчиненных объектов для любого головного объекта необходимо соблюдать. Если это требование исходя из входных данных нарушается, то соответствующий подчиненный объект не создается.

Добавить проверку допустимости исходной сборки. Собрать дерево невозможно, если по заданной координате головной объект не найден (например,

ошибка в наименовании или еще не расположен на дереве объектов). Если номер класса объекта задан некорректно, то объект не создается.

Собранная система обрабатывает следующие команды:

- SET «координата» – устанавливает текущий объект;
- FIND «координата» – находит объект относительно текущего;
- MOVE «координата» – переопределить головной для текущего объекта, «координата» задает новый головной объект;
- DELETE «наименование объекта» – удалить подчиненный объект у текущего;
- END – завершает функционирование системы (выполнение программы).

Изначально, корневой объект для системы является текущим. При вводе данных в названии команд ошибок нет. Если при переопределении головного объекта нарушается уникальность наименований подчиненных объектов для нового головного, переопределение не производится.

1.1 Описание входных данных

Состав и иерархия объектов строиться посредством ввода исходных данных. Ввод организован как в версии № 2 курсовой работы. Единственное различие. В строке ввода первым указано не наименование головного объекта, а абсолютный путь к нему.

После ввода состава дерева иерархии построчно вводятся команды:

- SET «координата» – установить текущий объект;
- FIND «координата» – найти объект относительно текущего;
- MOVE «координата» – переопределить головной для текущего объекта, «координата» соответствует новому головному объекту;

- DELETE «наименование объекта» – удалить подчиненный объект у текущего;
- END – завершить функционирование системы (выполнение программы).

Команды SET, FIND, MOVE и DELETE вводятся произвольное число раз.

Команда END присутствует обязательно.

Пример ввода иерархии дерева объектов:

```
rootela
/ object_1 3
/ object_2 2
/object_2 object_4 3
/object_2 object_5 4
/ object_3 3
/object_2 object_3 6
/object_1 object_7 5
/object_2/object_4 object_7 3
endtree
FIND object_2/object_4
SET /object_2
FIND //object_7
FIND object_4/object_7
FIND .
FIND .object_7
FIND object_4/object_7
MOVE .object_7
SET object_4/object_7
MOVE //object_1
MOVE /object_3
END
```

1.2 Описание выходных данных

Первая строка:

```
object tree
```

Со второй строки вывести иерархию построенного дерева как в работе версия №2.

При ошибке определения головного объекта, прекратить сборку, вывести иерархию уже построенного фрагмента дерева, со следующей строки сообщение:

The head object «координата головного объекта» is not found

и прекратить работу программы с кодом возврата 1.

Если при построении при попытке создания объекта обнаружен дубляж, то вывести:

«координата головного объекта» Dubbing the names of subordinate objects

Если дерево построено, то далее построчно вводятся команды.

Для команд SET если объект найден, то вывести:

Object is set: «имя объекта»

в противном случае:

The object was not found at the specified coordinate: «искомая координата объекта»

Для команд FIND вывести:

«искомая координата объекта» Object name: «наименование объекта»

Если объект не найден, то:

«искомая координата объекта» Object is not found

Для команд MOVE вывести:

New head object: «наименование нового головного объекта»

Если головной объект не найден, то:

«искомая координата объекта» Head object is not found

Если переопределить головной объект не удалось, то:

«искомая координата объекта» Redefining the head object failed

Если у нового головного объекта уже есть подчиненный с таким же именем, то вывести:

«искомая координата объекта» Dubbing the names of subordinate objects

При попытке переподчинения головного объекта к объекту на ветке, вывести:

«координата нового головного объекта»

Redefining the head object failed

Для команды DELETE:

Если подчиненный объект удален, то вывести:

The object «абсолютный путь удаленного объекта» has been deleted

Если объект не найден, то ничего не выводить.

После команды END с новой строки вывести:

Current object hierarchy tree

Со следующей строки вывести текущую иерархию дерева.

Пример вывода иерархии дерева объектов:

```
Object tree
rootela
  object_1
    object_7
  object_2
    object_4
      object_7
    object_5
    object_3
  object_3
object_2/object_4      Object name: object_4
Object is set: object_2
//object_7            Object is not found
object_4/object_7      Object name: object_7
.      Object name: object_2
.object_7              Object name: object_7
object_4/object_7      Object name: object_7
.object_7              Redefining the head object failed
Object is set: object_7
//object_1             Dubbing the names of subordinate objects
New head object: object_3
Current object hierarchy tree
rootela
  object_1
    object_7
  object_2
    object_4
    object_5
    object_3
  object_3
    object_7
```

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- библиотека `sstream` для работы со строками как с потоками ввода/вывода.

Класс `cl_base`:

- функционал:
 - метод `set_head` — метод переопределения головного объекта для текущего объекта;
 - метод `delete_subobject` — метод удаления подчинённого объекта по его наименованию;
 - метод `get_subobject_by_coordinate` — метод поиска объекта по координате (строковому пути) относительно текущего объекта или от корня.

Класс `cl_application`:

- функционал:
 - метод `build_tree_objects` — построение исходного дерева иерархии объектов на основе входных данных;
 - метод `exes_app` — запуск системы, обработка команд управления объектами.

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	cl_base				
		cl_application	public		2
2	cl_application				

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм метода `set_head` класса `cl_base`

Функционал: метод переопределения головного объекта для текущего объекта.

Параметры: указатель на объект базового класса, `new_head`, содержит адрес нового головного объекта.

Возвращаемое значение: булевое - истина, если головной объект успешно переопределён; ложь, если переопределение невозможно.

Алгоритм метода представлен в таблице 2.

Таблица 2 – Алгоритм метода `set_head` класса `cl_base`

№	Предикат	Действия	№ перехода
1	у текущего объекта нет головного или значение указателя <code>new_head</code> равно нулевому	вернуть false	Ø
			2
2		инициализировать указатель <code>ancestor</code> адресом нового головного объекта <code>new_head</code>	3
3	значение указателя <code>ancestor</code> не нулевое		4
			5
4	значение указателя <code>ancestor</code> совпадает с адресом	вернуть ложь	Ø

№	Предикат	Действия	№ перехода
	текущего объекта		
		присвоить указателю ancestor результат вызова метода get_head по указателю ancestor	3
5	у нового головного уже есть подчинённый с таким же именем	вернуть ложь	∅
			6
6		инициализировать целочисленную переменную i нулевым значением	7
7	значение индекса i меньше количества подчинённых объектов у головного объекта текущего объекта		8
			9
8	значение i-го элемента из списка адресов подчинённых объектов текущего головного объекта совпадает с адресом текущего объекта	удалить адрес текущего объекта из списка адресов подчинённых объектов головного объекта	9
		инкрементировать i	7
9		присвоить закрытому полю p_head_object текущего объекта значение входного параметра new_head	10
10		добавить адрес текущего объекта в список адресов подчинённых объектов нового головного объекта	11
11		вернуть истину	∅

3.2 Алгоритм метода delete_subobject класса cl_base

Функционал: метод удаления подчинённого объекта по его наименованию.

Параметры: строковый, name, наименование удаляемого подчинённого объекта.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода delete_subobject класса cl_base

№	Предикат	Действия	№ перехода
1		инициализировать целочисленную переменную i нулевым значением	2
2	значение индекса i меньше количества подчинённых объектов текущего объекта		3
			∅
3	имя i-го элемента массива подчинённых объектов которого хранится в i-том элементе массива равно входному параметру name	удалить динамически созданный объект, адрес которого хранится в i-том элементе массива подчинённых объектов текущего объекта	4
		инкрементировать i	2
4		удалить i-ый элемент из массива адресов подчинённых объектов текущего объекта	∅

3.3 Алгоритм метода get_subobject_by_coordinate класса cl_base

Функционал: метод поиска объекта по координате (строковому пути) относительно текущего объекта или от корня.

Параметры: строковый, coordinate, путь для поиска объекта.

Возвращаемое значение: адрес найденного объекта (nullptr, если объект не найден или координата некорректна).

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода *get_subobject_by_coordinate* класса *cl_base*

№	Предикат	Действия	№ перехода
1	строка coordinate пустая	вернуть значение нулевого указателя	Ø
		инициализировать указатель root адресом текущего объекта	2
2	значение параметра coordinate равно "."	вернуть адрес текущего объекта	Ø
			3
3	у объекта, на который указывает root, есть головной объект	присвоить указателю root результат вызова метода get_head по указателю root	3
			4
4	значение параметра coordinate равно "/"	вернуть значение указателя root	Ø
			5
5	длина строки coordinate не меньше 2 и первые два символа равны "/"	вернуть результат вызова метода get_subobject_on_tree по указателю root с частью строки coordinate без первых двух символов в качестве аргумента	Ø
			6
6	первый символ строки coordinate равен "."	вернуть результат вызова метода get_subobject_on_branch текущего объекта с частью строки coordinate без первого символа в качестве аргумента	Ø
			7
7		создать объект ss класса stringstream, передав в	8

№	Предикат	Действия	№ перехода
		качестве аргумента строку coordinate	
8		объявить строковую переменную part	9
9		объявить указатель на объект базового класса cur	10
10	первый символ строки coordinate равен "/"	вызов метода ss.get() для пропуска первого слэша	11
		присвоить указателю cur адрес текущего объекта	12
11		присвоить указателю cur значение указателя root	12
12	из потока ss можно извлечь часть строки в переменную part по разделителю '/'		13
			15
13	строка part пустая		15
		инициализировать указатель на объект базового класса found результатов вызова метода get_subobject_by_name по указателю cur с аргументом part	14
14	значение указателя found нулевое	вернуть значение нулевого указателя	∅
		присвоить указателю cur значение указателя found	12
15		вернуть адрес найденного объекта cur	∅

3.4 Алгоритм метода build_tree_objects класса cl_application

Функционал: построение исходного дерева иерархии объектов на основе входных данных.

Параметры: нет.

Возвращаемое значение: отсутствует.

Алгоритм метода представлен в таблице 5.

Таблица 5 – Алгоритм метода *build_tree_objects* класса *cl_application*

№	Предикат	Действия	№ перехода
1		объявить строковые переменные s_coord и s_sub	2
2		объявить целочисленную переменную class_number	3
3		считать значение из стандартного потока ввода в переменную s_coord	4
4		вызов метода редактирования имени set_name текущего объекта с аргументом s_coord	5
5		считать значение из стандартного потока ввода в переменную s_coord	6
6	значение переменной s_coord равно "endtree"		∅
		считать значение из стандартного потока ввода в переменные s_sub и class_number	7
7		инициализировать указатель на объект базового класса p_head адресом предполагаемого головного объекта посредством вызова метода get_subobject_by_coordinate с аргументами s_coord и адресом текущего объекта	8
8	предполагаемый головной объект не был найден?	вывод на консоль строки "Object tree" и символа переноса строки	9
			12
9		вызов метода print_tree текущего объекта	10
10		вывод на консоль строки "The head object ", значения переменной s_coord, строки " is not found" и символа переноса строки	11
11		установить состояние готовности корневого объекта на 1 как признак ошибки посредством вызова метода set_ready с аргументом 1	∅
12	результат вызова метода	вывод на консоль значения переменной s_coord,	5

№	Предикат	Действия	№ перехода
	get_subobject_by_name по строки " Dubbing the names of subordinate указателю p_head с objects" и символа переноса строки аргументом s_sub не равен значению нулевого указателя		
			13
13	номер класса меньше 2 или больше 6		5
			14
14	значение переменной class_number равно 2	динамически создать объект класса cl_2 с аргументами p_head, s_sub	5
	значение переменной class_number равно 3	динамически создать объект класса cl_3 с аргументами p_head, s_sub	5
	значение переменной class_number равно 4	динамически создать объект класса cl_4 с аргументами p_head, s_sub	5
	значение переменной class_number равно 5	динамически создать объект класса cl_5 с аргументами p_head, s_sub	5
	значение переменной class_number равно 6	динамически создать объект класса cl_6 с аргументами p_head, s_sub	5
			5

3.5 Алгоритм метода exes_app класса cl_application

Функционал: запуск системы, обработка команд управления объектами.

Параметры: нет.

Возвращаемое значение: целочисленный - код возврата программы, определяющий корректность выполнения работы.

Алгоритм метода представлен в таблице 6.

Таблица 6 – Алгоритм метода *exec_app* класса *cl_application*

№	Предикат	Действия	№ перехода
1	состояние готовности корневого объекта равно 1	вернуть 0	Ø
			2
2		вывод на консоль строки "Object tree" и символа переноса строки	3
3		вызов метода <code>print_tree</code> текущего объекта	4
4		инициализировать указатель на объекта базового класса <code>current_object</code> адресом текущего объекта	5
5		объявить строковую переменную <code>command</code>	6
6		объявить строковую переменную <code>param</code>	7
7		считать значение из стандартного потока ввода в переменную <code>command</code>	8
8	значение переменной <code>command</code> равно "END"		25
		считать значение из стандартного потока ввода в переменную <code>param</code>	9
9	значение переменной <code>command</code> равно "SET"	инициализировать указатель на объект базового класса <code>found</code> результатом вызова метода <code>get_subobject_by_coordinate</code> по указателю <code>current_object</code> с аргументом <code>param</code>	10
	значение переменной <code>command</code> равно "FIND"	инициализировать указатель на объект базового класса <code>found</code> результатом вызова метода <code>get_subobject_by_coordinate</code> по указателю <code>current_object</code> с аргументом <code>param</code>	12
	значение переменной <code>command</code> равно "MOVE"	инициализировать указатель на объект базового класса <code>new_head</code> результатом вызова метода <code>get_subobject_by_coordinate</code> по указателю <code>current_object</code> с аргументом <code>param</code>	13
	значение переменной	инициализировать указатель на объект базового	18

№	Предикат	Действия	№ перехода
	command равно "DELETE"	класса to_delete результатом вызова метода get_subobject_by_name по указателю current_object с аргументом param	
			25
10	значение указателя found не нулевое	присвоить указателю current_object значение указателя found	11
		вывод на консоль строки "The object was not found at the specified coordinate: ", значения переменной param и символа переноса строки	7
11		вывод на консоль строки "Object is set: ", результата вызова метода get_name по указателю found и символа переноса строки	7
12	значение указателя found не нулевое	вывод на консоль значения переменной param, строки " Object name: ", результата вызова метода get_name по указателю found и символа переноса строки	7
		вывод на консоль значения переменной param, строки " Object is not found" и символа переноса строки	7
13	значение указателя new_head не нулевое	вывод на консоль значения переменной param, строки " Head object is not found" и символа переноса строки	7
		инициализировать указатель на объект базового класса temp значением указателя new_head	14
14	значение указателя temp не нулевое		15
			16

№	Предикат	Действия	№ перехода
15	значение указателя temp совпадает со значением указателя current_object	вывод на консоль значения переменной param, строки " Redefining the head object failed" и символа переноса строки	16
		присвоить указателю temp результат вызова метода get_head по указателю temp	14
16	значение указателя temp не нулевое		7
			17
17	результат вызова метода get_subobject_by_name по указателю new_head, передавая результат вызова метода get_name по указателю current_object в качестве аргумента, не равен значению нулевого указателя	вывод на консоль значения переменной param, строки " Dubbing the names of subordinate objects" и символа переноса строки	7
	результат вызова метода set_name по указателю current_object с аргументом new_name равен истине	вывод на консоль строки "New head object: ", результата вызова метода get_name по указателю new_head и символа переноса строки	7
		вывод на консоль значения переменной param, строки " Redefining the head object failed" и символа переноса строки	7
18	значение указателя to_delete не нулевое	объявить строковую переменную abs_path	19
			7
19		инициализировать указатель на объект базового класса path_builder значением указателя to_delete	20

№	Предикат	Действия	№ перехода
20	значение указателя path_builder не нулевое		21
			23
21	результат вызова метода get_head по указателю path_builder не равен нулевому указателю	присвоить переменной abs_path результат операции "/" + результат вызова метода get_name по указателю path_builder + abs_path	22
			22
22		присвоить переменной path_builder результат вызова метода get_head по указателю path_builder	20
23		вызов метода delete_subobject по указателю current_object с аргументом param	24
24		вывод на консоль строки "The object ", значения переменной abs_path, строки " has been deleted" и символа переноса строки	7
25		вывод на консоль строки "Current object hierarchy tree" и символа переноса строки	26
26		вызов метода print_tree текущего объекта	27
27		вернуть 0	∅

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-16.

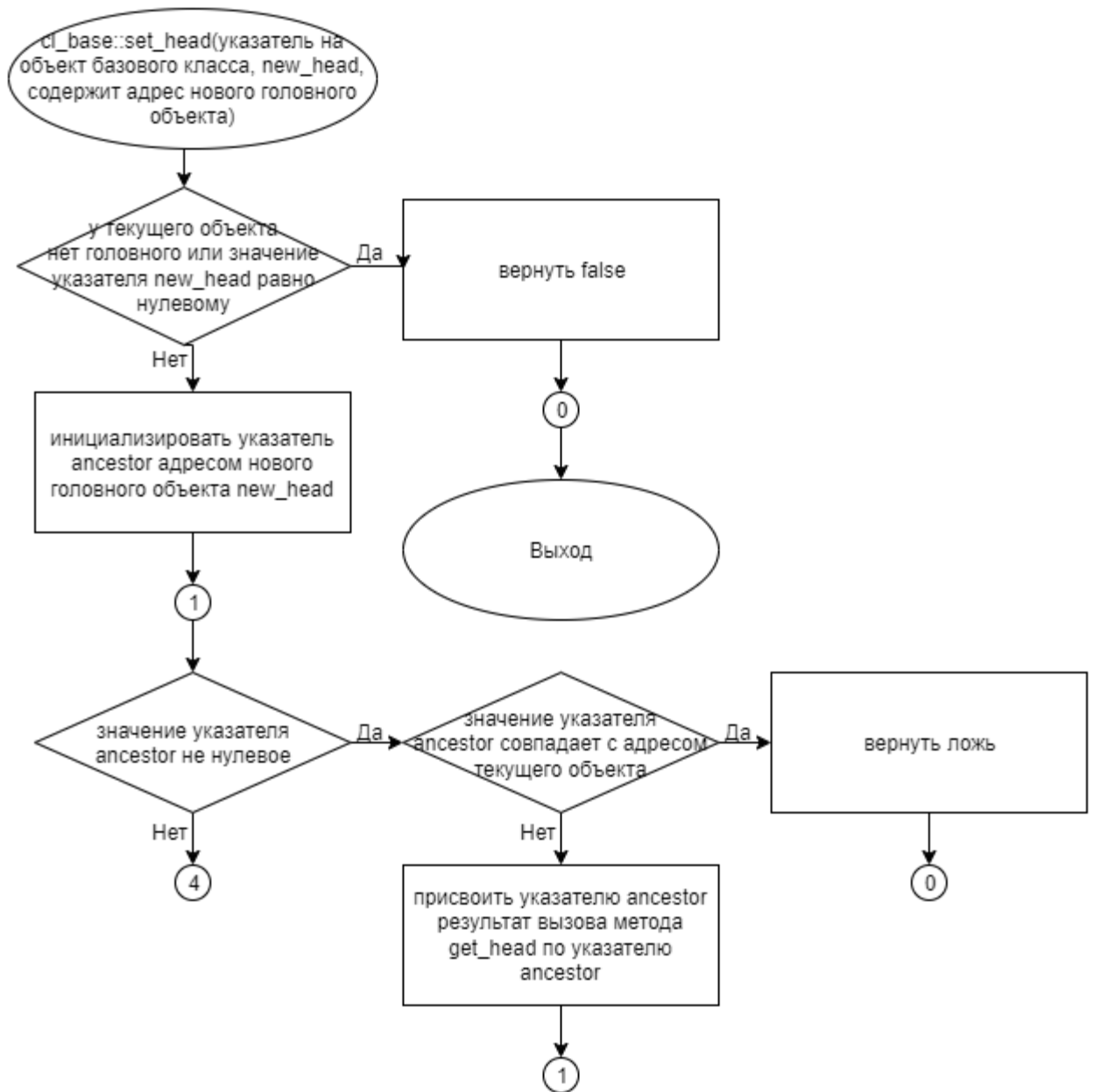


Рисунок 1 – Блок-схема алгоритма

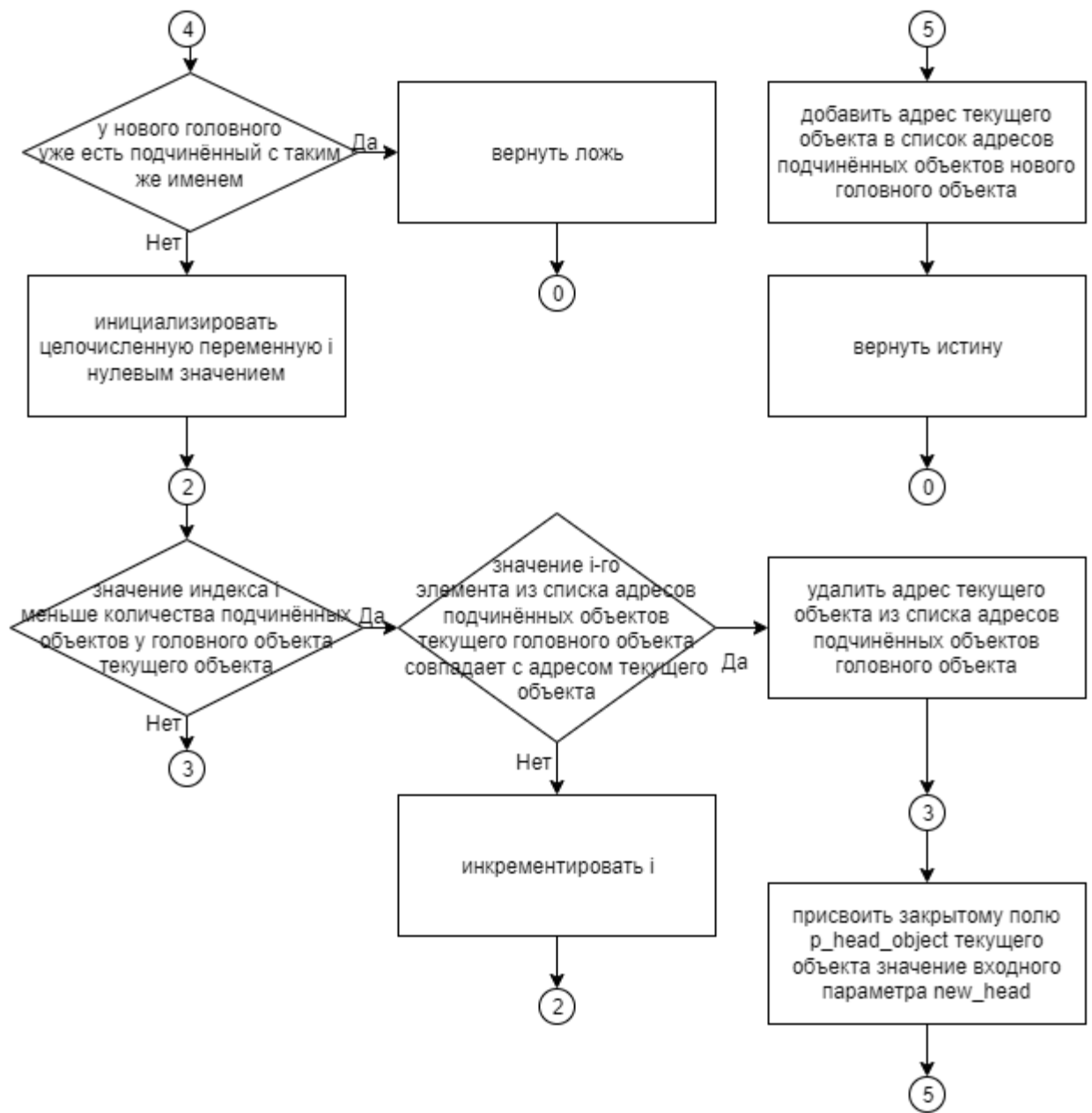


Рисунок 2 – Блок-схема алгоритма

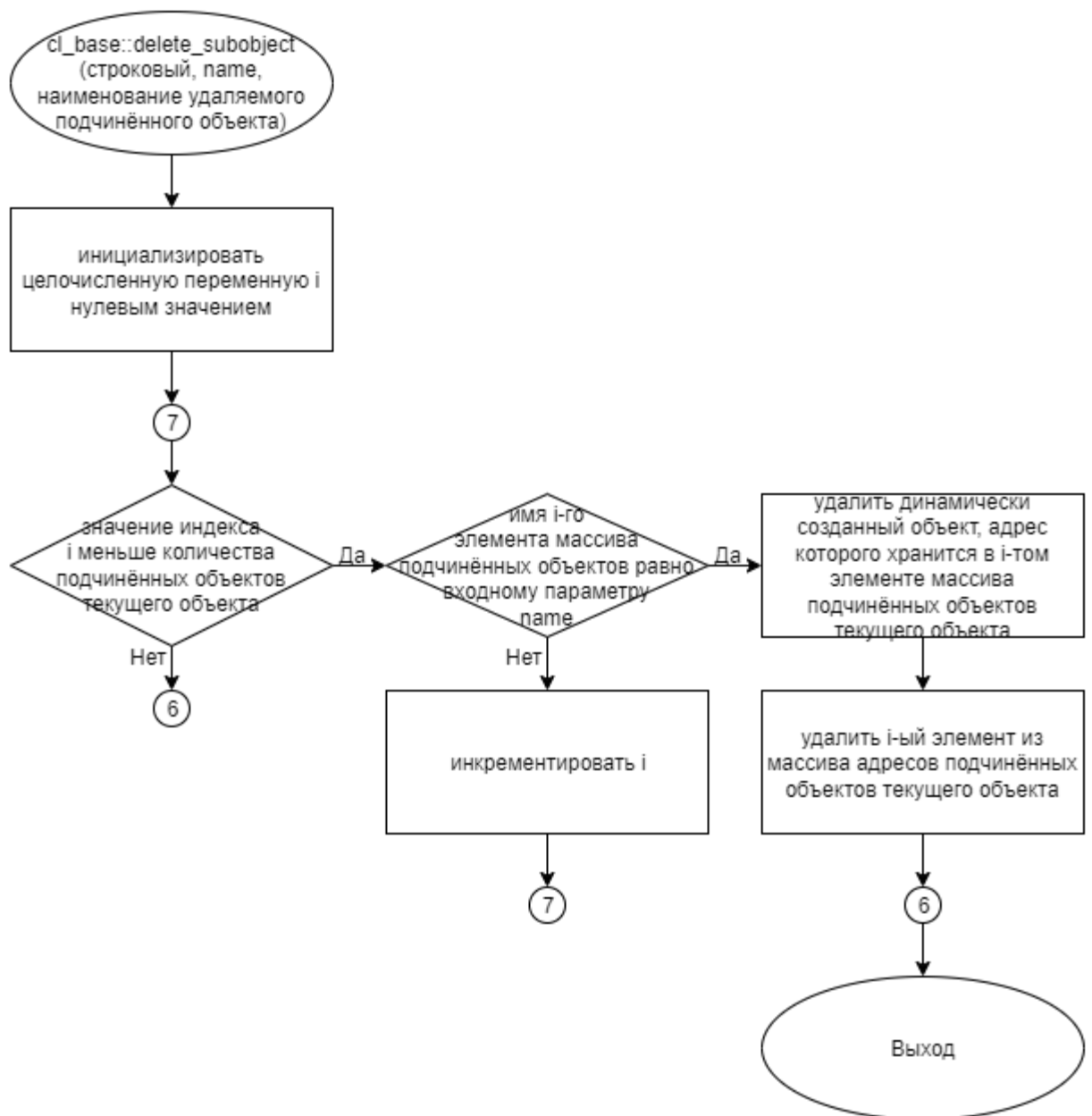


Рисунок 3 – Блок-схема алгоритма

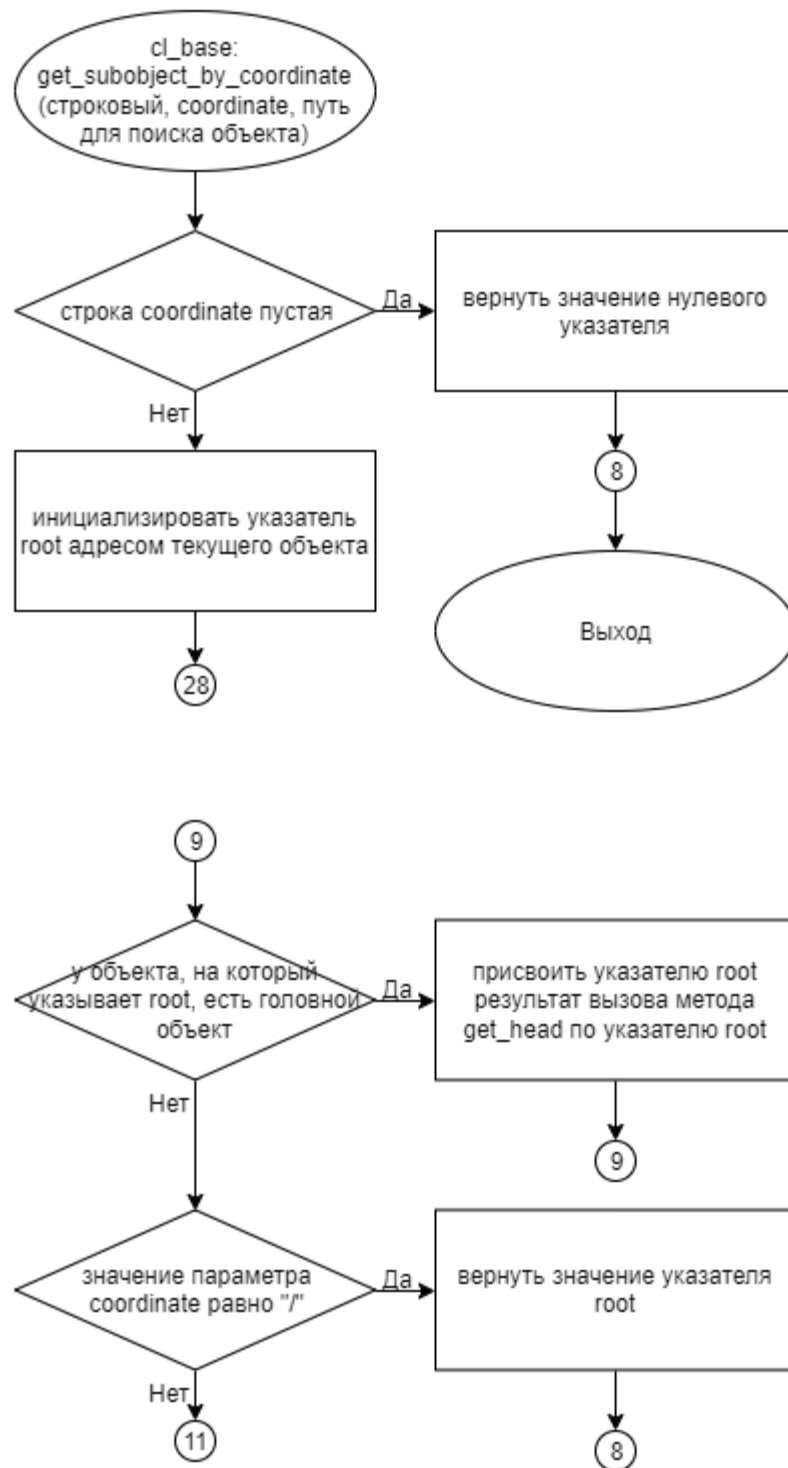


Рисунок 4 – Блок-схема алгоритма

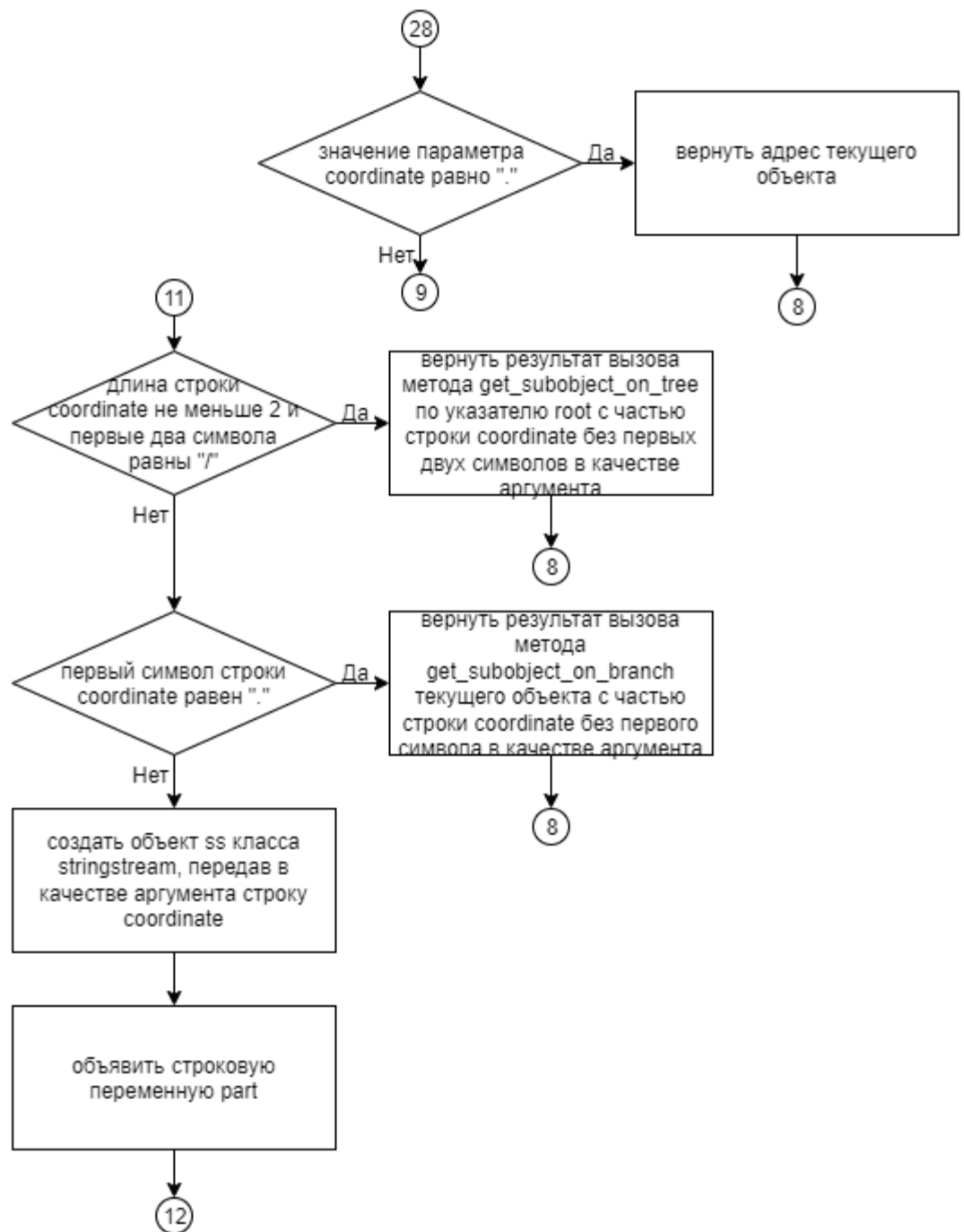


Рисунок 5 – Блок-схема алгоритма

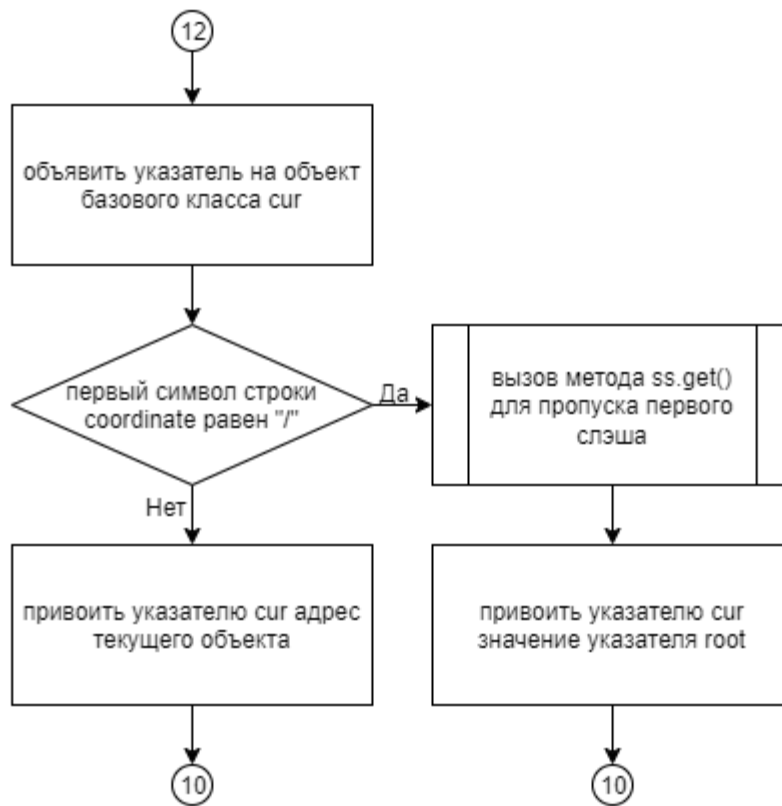


Рисунок 6 – Блок-схема алгоритма

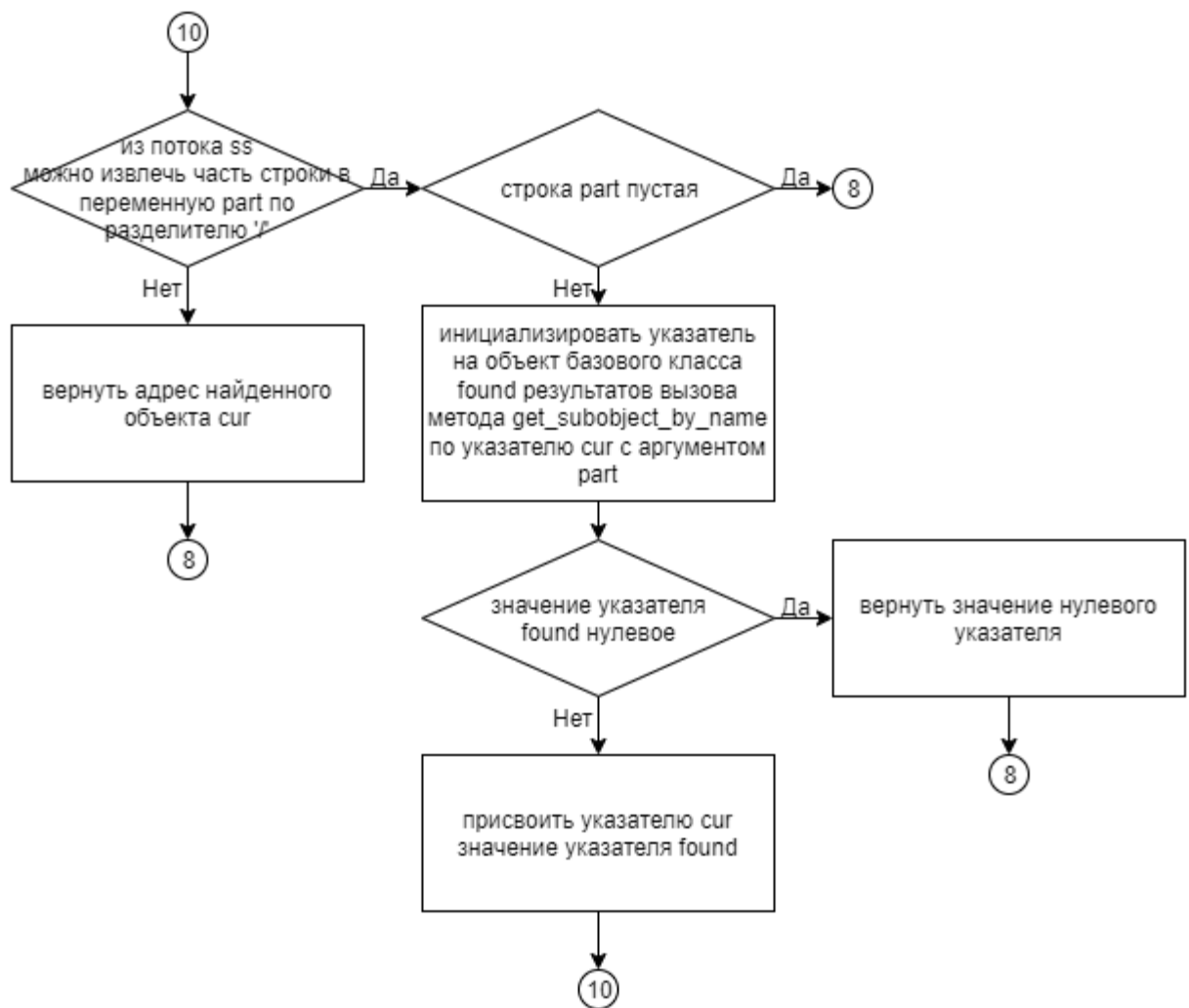


Рисунок 7 – Блок-схема алгоритма

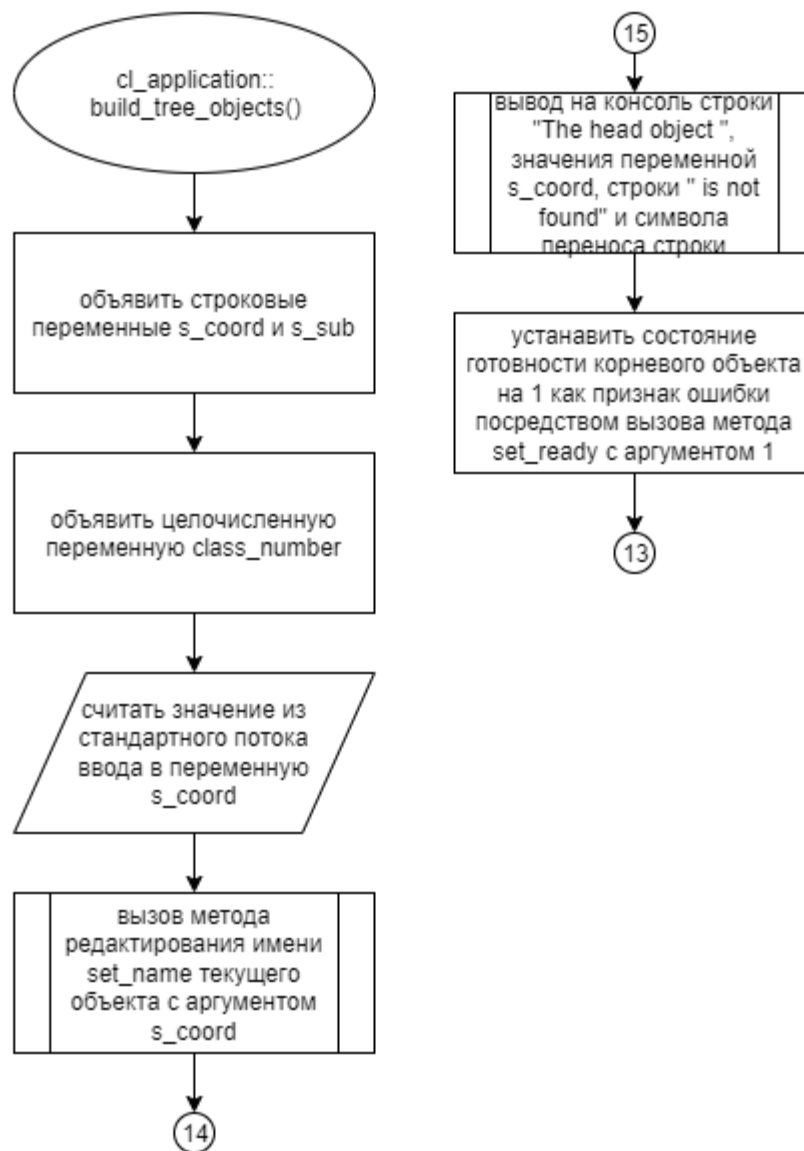


Рисунок 8 – Блок-схема алгоритма

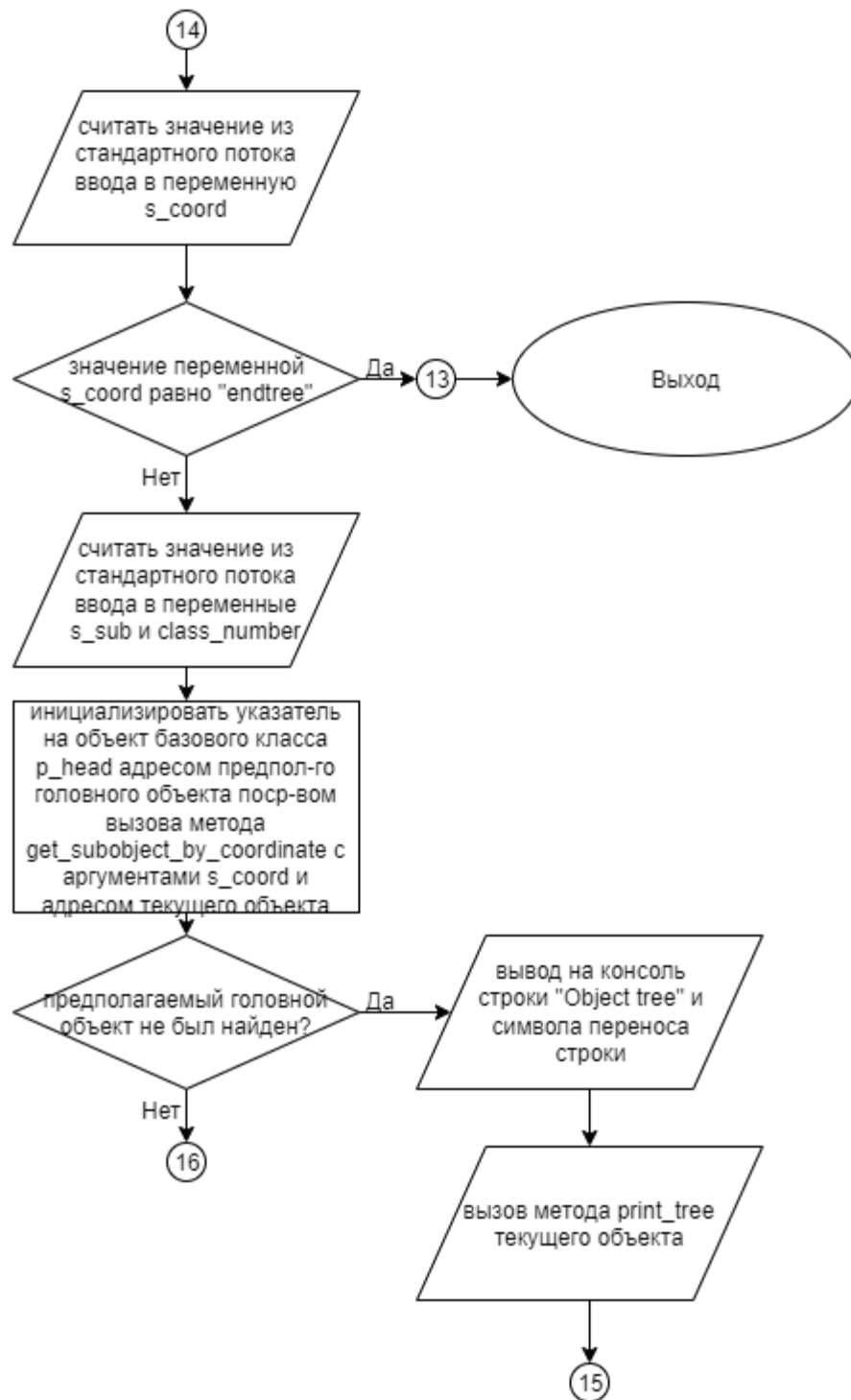


Рисунок 9 – Блок-схема алгоритма

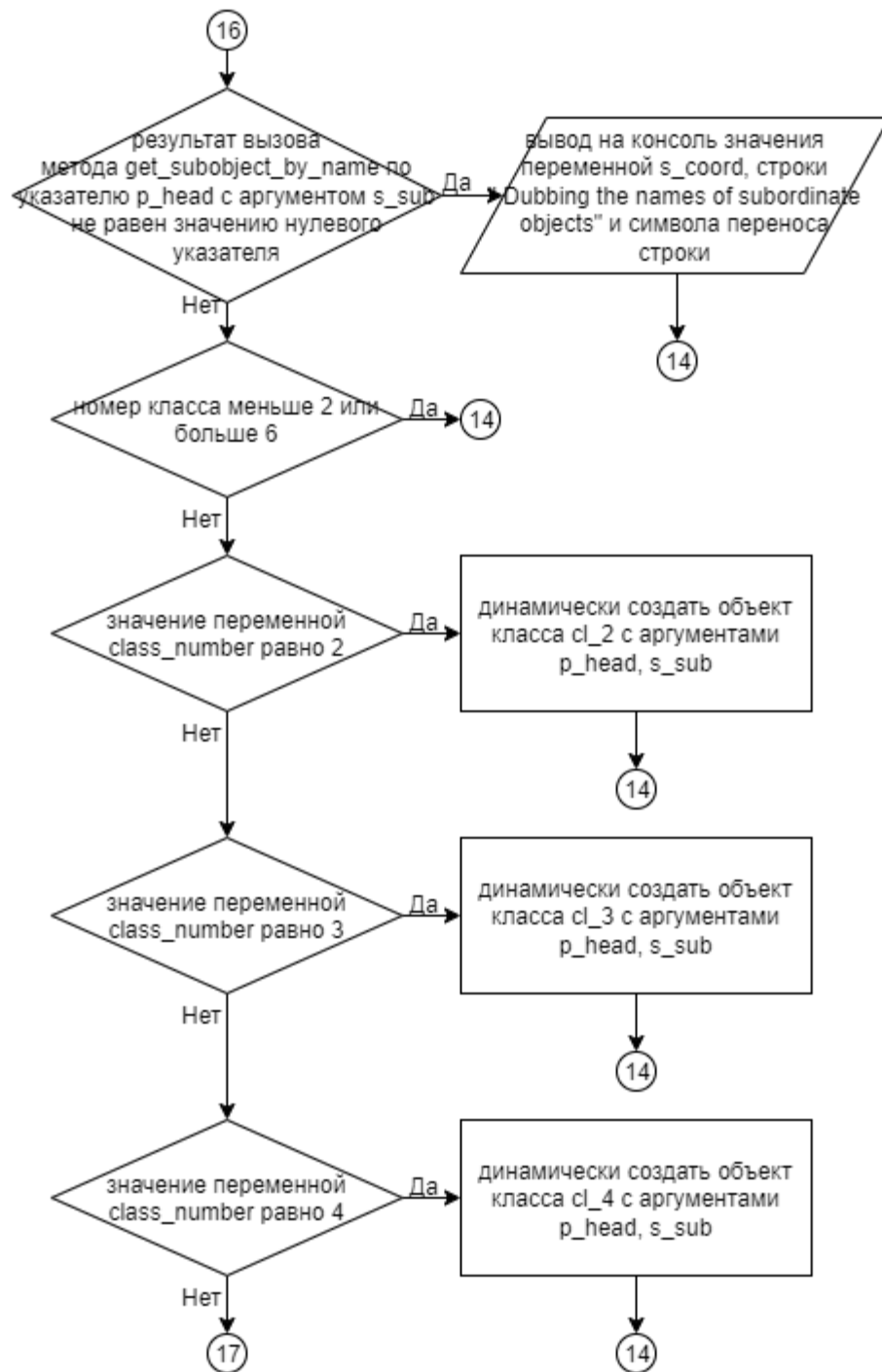


Рисунок 10 – Блок-схема алгоритма

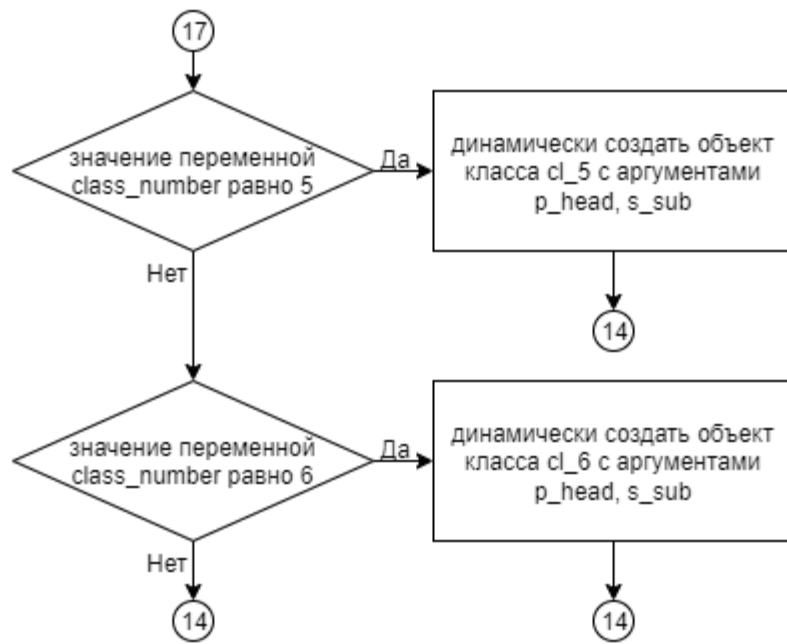


Рисунок 11 – Блок-схема алгоритма

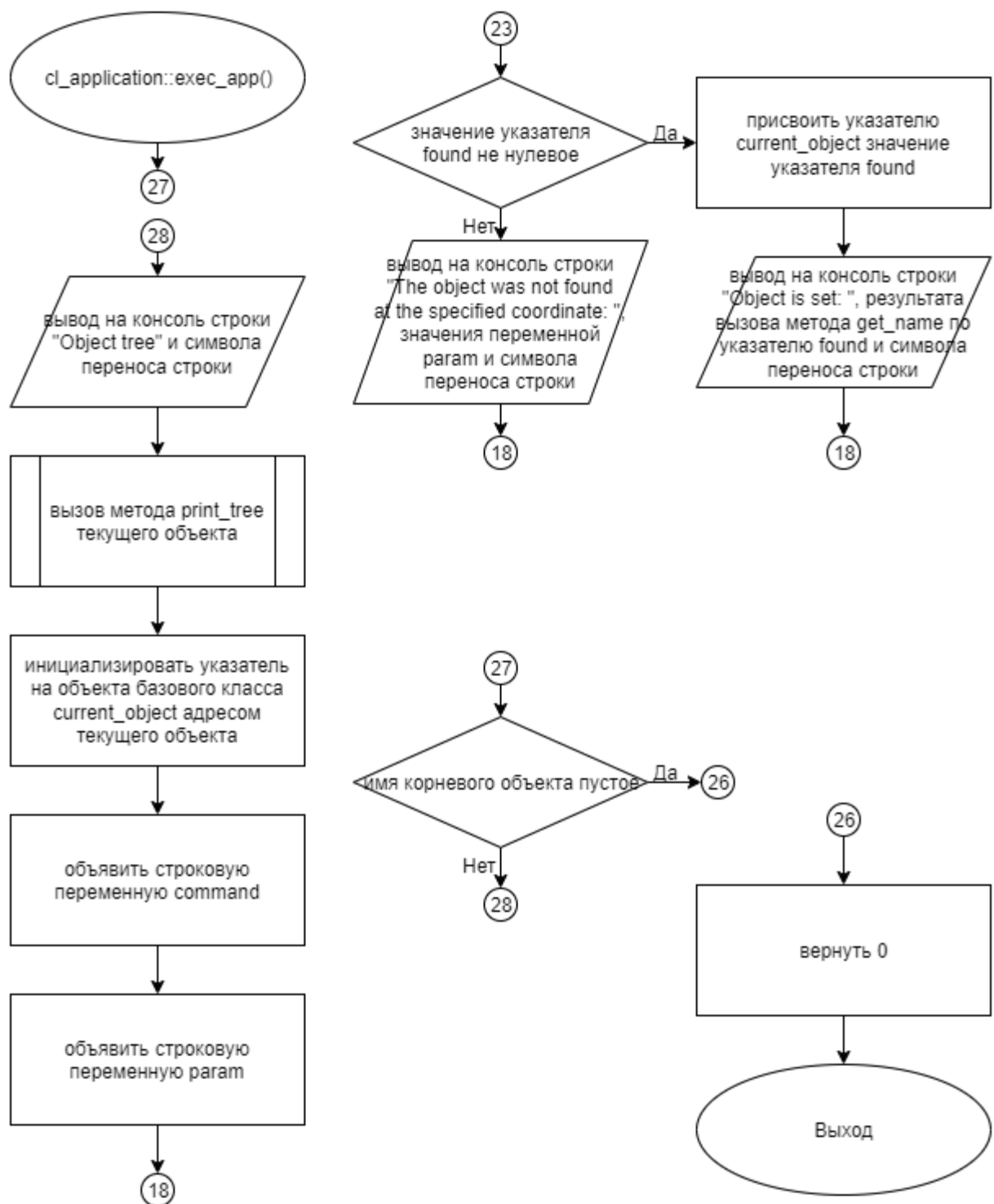


Рисунок 12 – Блок-схема алгоритма

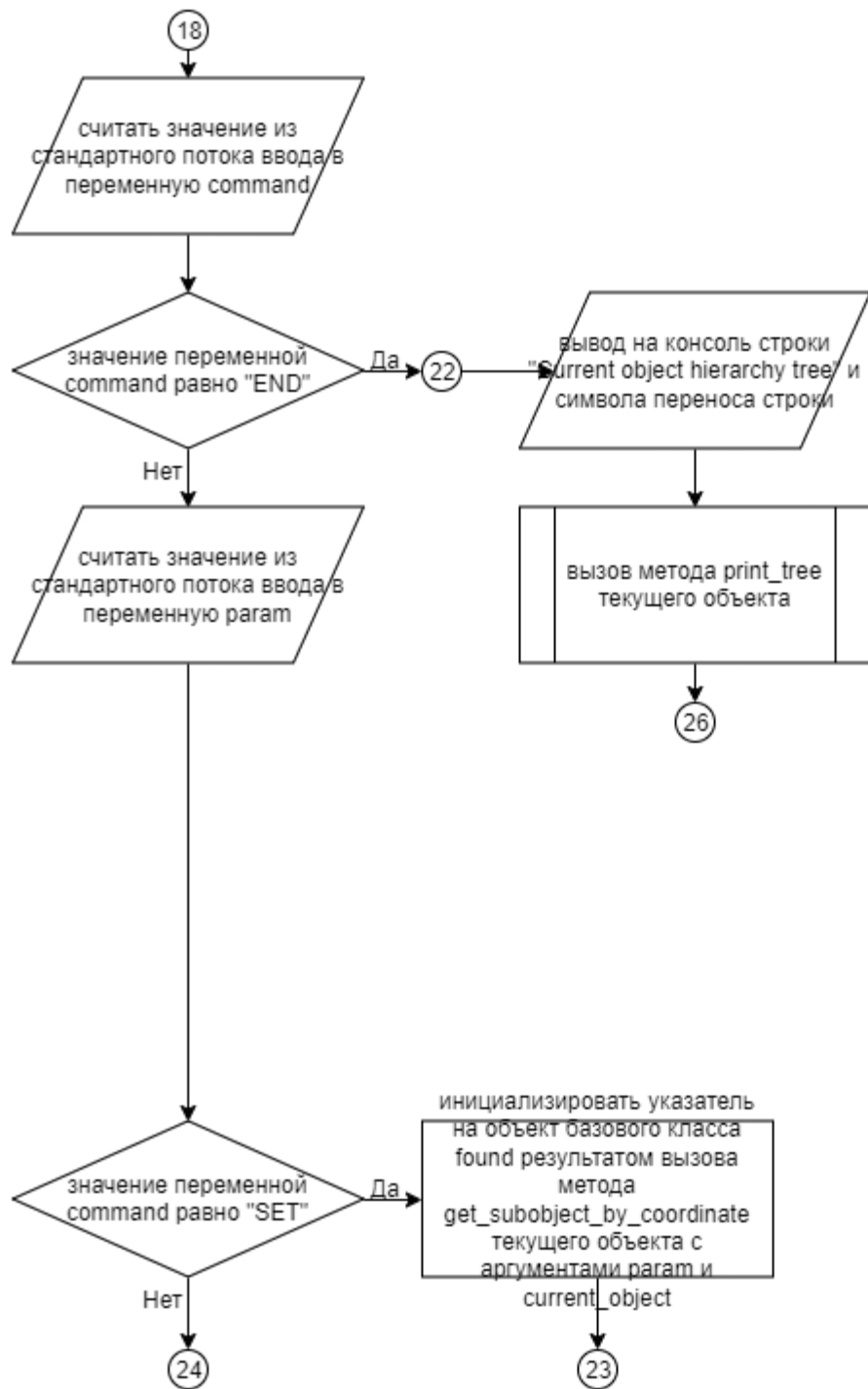


Рисунок 13 – Блок-схема алгоритма

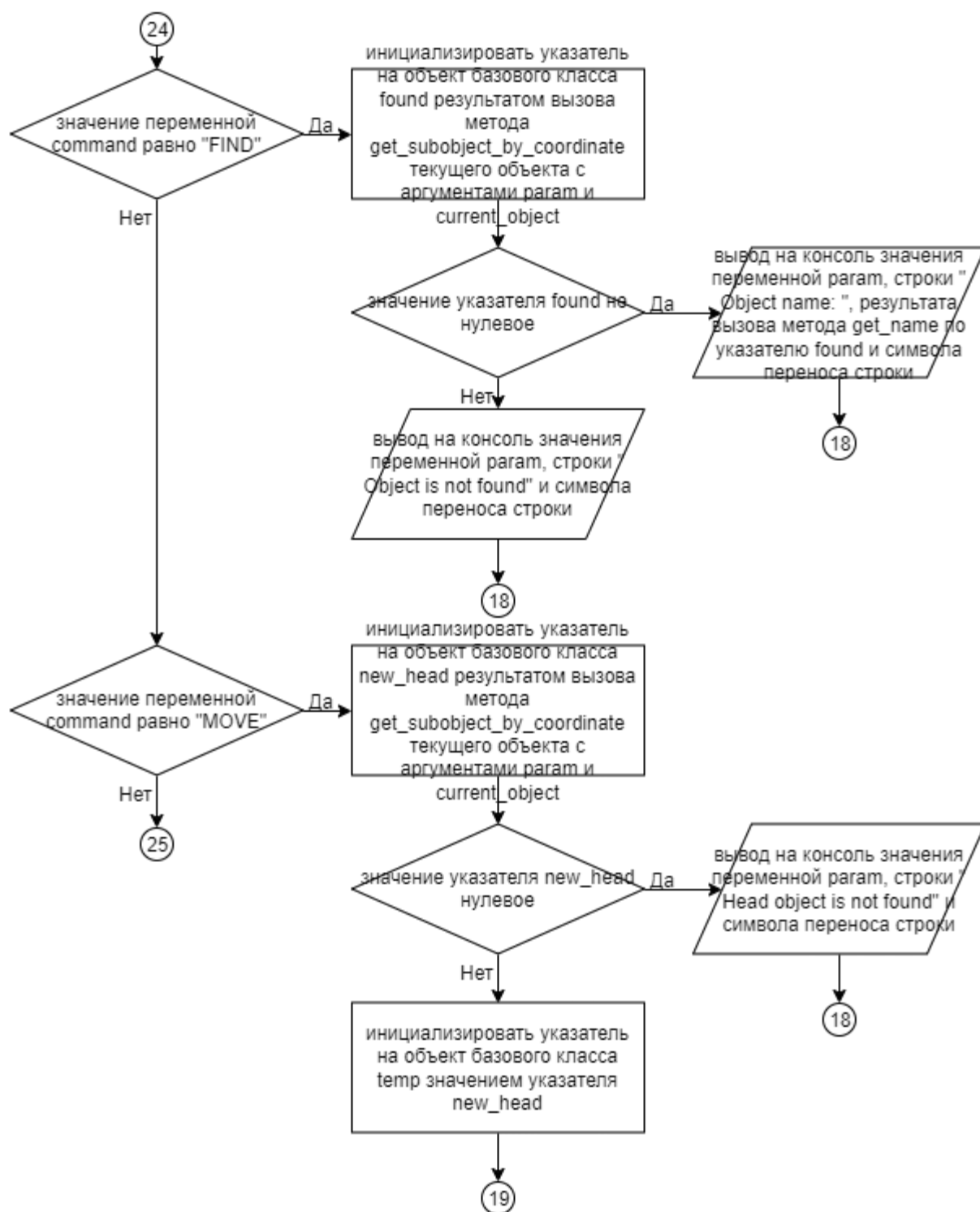


Рисунок 14 – Блок-схема алгоритма

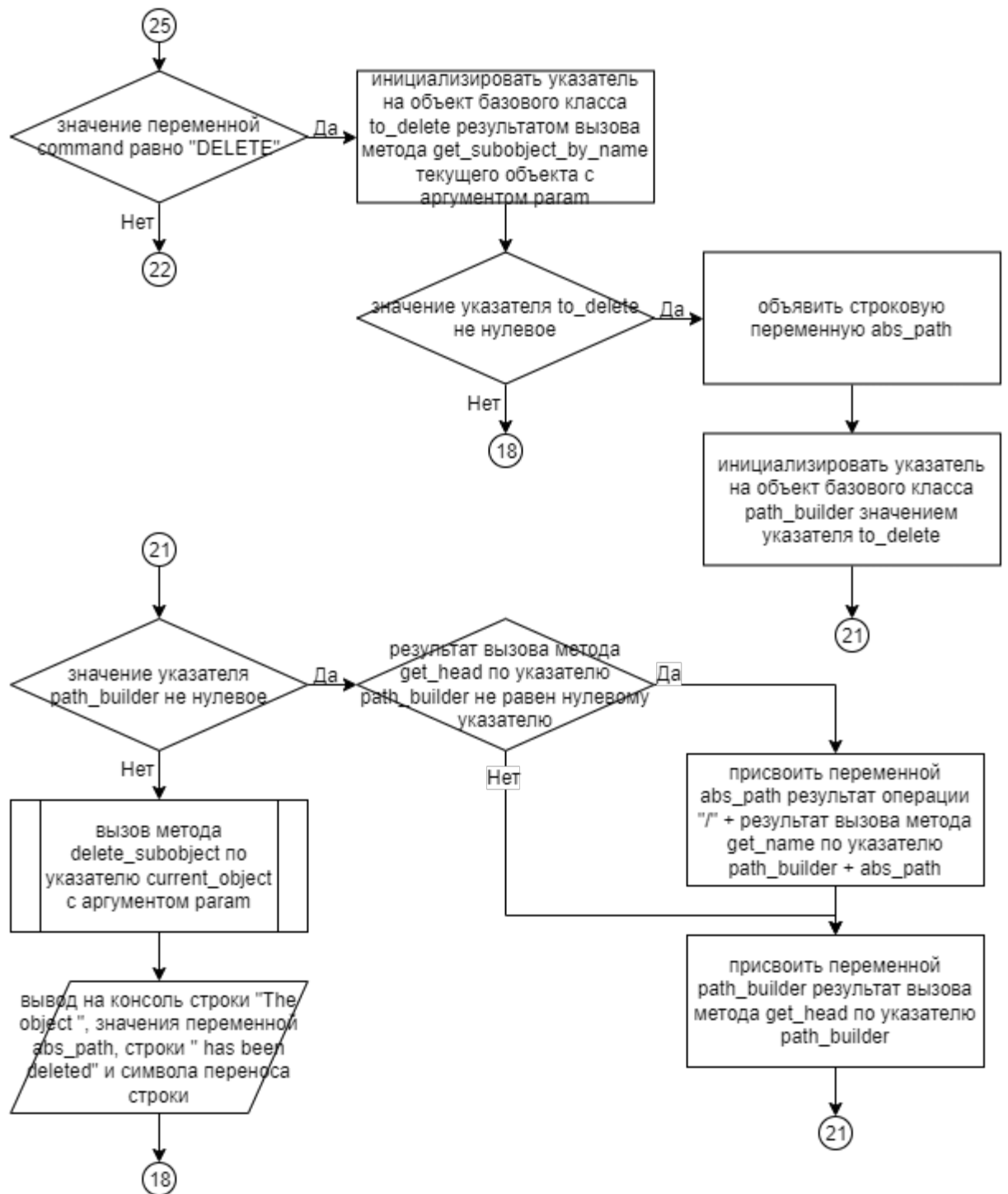


Рисунок 15 – Блок-схема алгоритма

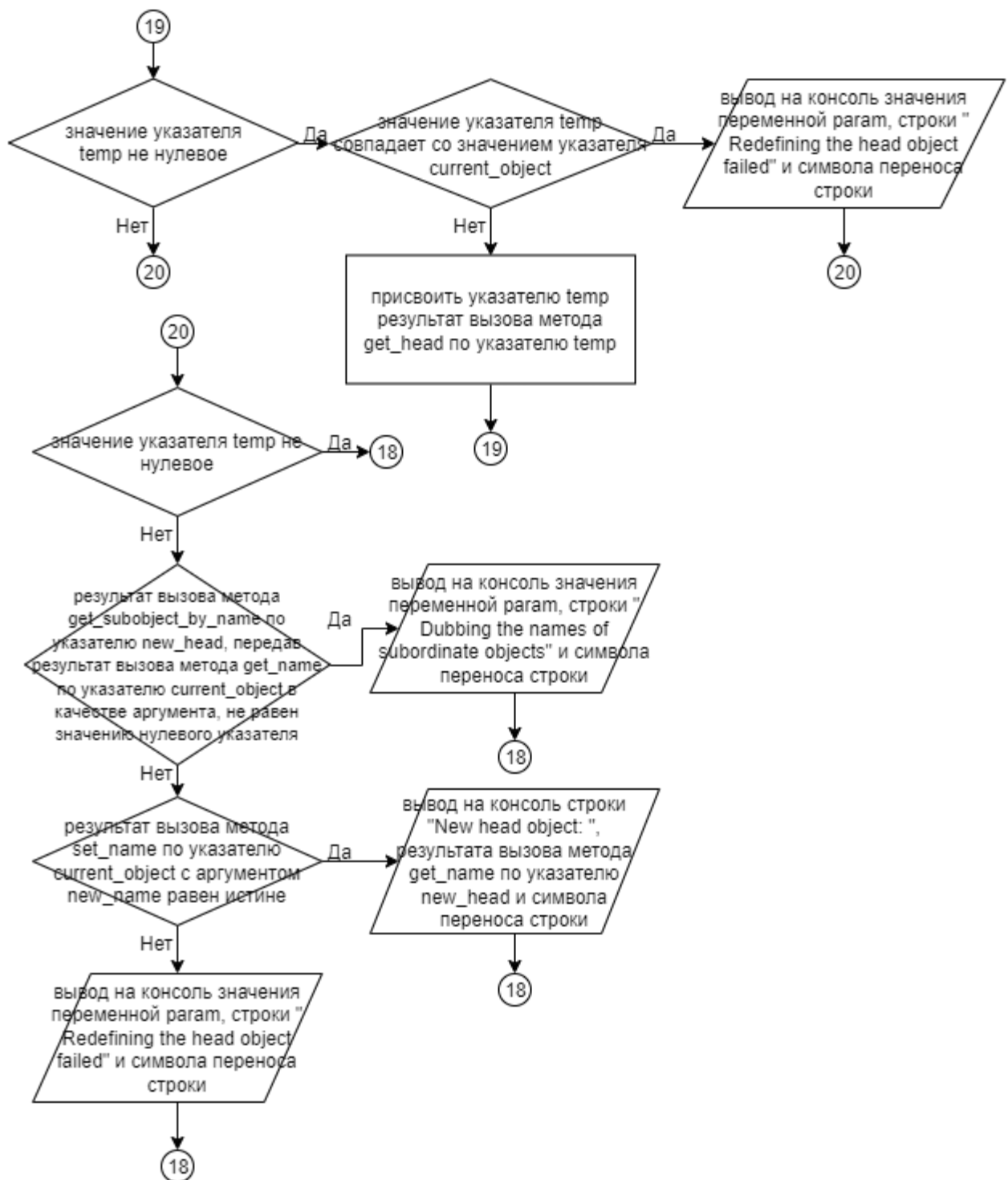


Рисунок 16 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл cl_2.cpp

Листинг 1 – cl_2.cpp

```
#include "cl_2.h"

cl_2 :: cl_2 ( cl_base * p_head_object, string s_object_name )
    : cl_base ( p_head_object, s_object_name ) { }    // конструирует объект
производного класса с указателем на головной объект и наименованием путём
вызова конструктора базового класса
```

5.2 Файл cl_2.h

Листинг 2 – cl_2.h

```
#ifndef __CL_2__H
#define __CL_2__H

#include "cl_base.h"

class cl_2 : public cl_base
{
public:
    cl_2 ( cl_base * p_head_object, string s_object_name );    //
параметризированный конструктор
};

#endif
```

5.3 Файл cl_3.cpp

Листинг 3 – cl_3.cpp

```
#include "cl_3.h"
```

```
cl_3 :: cl_3 ( cl_base * p_head_object, string s_object_name )  
    : cl_base ( p_head_object, s_object_name ) { }      // конструирует объект  
производного класса с указателем на головной объект и наименованием путём  
вызова конструктора базового класса
```

5.4 Файл cl_3.h

Листинг 4 – cl_3.h

```
#ifndef __CL_3__H  
#define __CL_3__H  
  
#include "cl_base.h"  
  
class cl_3 : public cl_base  
{  
public:  
    cl_3 ( cl_base * p_head_object, string s_object_name ); //  
параметризованный конструктор  
};  
  
#endif
```

5.5 Файл cl_4.cpp

Листинг 5 – cl_4.cpp

```
#include "cl_4.h"  
  
cl_4 :: cl_4 ( cl_base * p_head_object, string s_object_name )  
    : cl_base ( p_head_object, s_object_name ) { }      // конструирует объект  
производного класса с указателем на головной объект и наименованием путём  
вызова конструктора базового класса
```

5.6 Файл cl_4.h

Листинг 6 – cl_4.h

```
#ifndef __CL_4__H
#define __CL_4__H

#include "cl_base.h"

class cl_4 : public cl_base
{
public:
    cl_4 (    cl_base *    p_head_object,    string    s_object_name    );    //
параметризированный конструктор
};

#endif
```

5.7 Файл cl_5.cpp

Листинг 7 – cl_5.cpp

```
#include "cl_5.h"

cl_5 :: cl_5 ( cl_base * p_head_object, string s_object_name )
    : cl_base ( p_head_object, s_object_name ) { }    // конструирует объект
производного класса с указателем на головной объект и наименованием путём
вызова конструктора базового класса
```

5.8 Файл cl_5.h

Листинг 8 – cl_5.h

```
#ifndef __CL_5__H
#define __CL_5__H

#include "cl_base.h"

class cl_5 : public cl_base
{
public:
    cl_5 (    cl_base *    p_head_object,    string    s_object_name    );    //
параметризированный конструктор
```

```
};  
  
#endif
```

5.9 Файл cl_6.cpp

Листинг 9 – cl_6.cpp

```
#include "cl_6.h"  
  
cl_6 :: cl_6 ( cl_base * p_head_object, string s_object_name )  
    : cl_base ( p_head_object, s_object_name ) { } // конструирует объект  
производного класса с указателем на головной объект и наименованием путём  
вызова конструктора базового класса
```

5.10 Файл cl_6.h

Листинг 10 – cl_6.h

```
#ifndef __CL_6__H  
#define __CL_6__H  
  
#include "cl_base.h"  
  
class cl_6 : public cl_base  
{  
public:  
    cl_6 ( cl_base * p_head_object, string s_object_name ); //  
параметризированный конструктор  
};  
  
#endif
```

5.11 Файл cl_application.cpp

Листинг 11 – cl_application.cpp

```
#include "cl_application.h"  
#include "cl_2.h"
```

```

#include "cl_3.h"
#include "cl_4.h"
#include "cl_5.h"
#include "cl_6.h"
#include <iostream>
#include <string>
#include <vector>
using namespace std;

cl_application :: cl_application ( cl_base * p_head_object, string
s_object_name )
: cl_base ( p_head_object, s_object_name ) { } // создаёт корневой
объект приложения, предварительно вызывая конструктор базового класса

void cl_application :: build_tree_objects ( ) // строит исходное
дерево иерархии объектов
{
    string s_coord, s_sub; // координата головного
объекта и имя подчинённого объектов
    int class_number; // объявл. целочисленной
переменной class_number

    cin >> s_coord; // считать значение из
стандартного потока ввода в переменную s_coord
    this -> set_name ( s_coord ); // устанавливаем имя
текущего объекта значением s_head вызовом метода set_name

    while ( true ) // начало бесконечного цикла
    {
        cin >> s_coord; // считать значение из
стандартного потока ввода в переменную s_coord

        if ( s_coord == "endtree" ) // значение переменной
s_coord равно "endtree"
        {
            break; // прервать выполнение цикла
- завершение построения дерева
        }

        cin >> s_sub >> class_number; // считать значение из
стандартного потока ввода в переменные s_sub и class_number

        cl_base * p_head = this -> get_subobject_by_coordinate ( s_coord );//
поиск головного по координате
        if ( p_head == nullptr ) // головной объект не найден
        {
            cout << "Object tree" << endl;
            this -> print_tree ( );
            cout << "The head object " << s_coord << " is not found" << endl;
            this -> set_ready ( 1 ); // устанавливаем состояние
готовности корневого объекта на 1 как признак ошибки
            return;
        }

        if ( p_head -> get_subobject_by_name ( s_sub ) != nullptr ) //

```

```

        проверка дуближа
        {
            // на дереве уже есть объект с
            именем создаваемого
            cout << s_coord << "      Dubbing the names of subordinate objects"
            << endl;
            continue; // обнаружен дуближ имени на
            дереве - объект не создаётся, переход к следующей итерации
        }

        if ( class_number < 2 || class_number > 6 ) // номер класса введён
        некорректно ( допустимые значения от 2 до 6)
        {
            continue; // номер задан некорректно
        }

        if ( class_number == 2 ) // создание объекта
        соответствующего класса
        {
            new cl_2 ( p_head, s_sub ); // динамически создать
            объект класса cl_2 с введёнными значениями в качестве аргументов
        }
        else if ( class_number == 3 )
        {
            new cl_3 ( p_head, s_sub ); // динамически создать
            объект класса cl_3 с введёнными значениями в качестве аргументов
        }
        else if ( class_number == 4 )
        {
            new cl_4 ( p_head, s_sub ); // динамически создать
            объект класса cl_4 с введёнными значениями в качестве аргументов
        }
        else if ( class_number == 5 )
        {
            new cl_5 ( p_head, s_sub ); // динамически создать
            объект класса cl_5 с введёнными значениями в качестве аргументов
        }
        else if ( class_number == 6 )
        {
            new cl_6 ( p_head, s_sub ); // динамически создать
            объект класса cl_6 с введёнными значениями в качестве аргументов
        }
    }
}

int cl_application :: exec_app ( ) // запускает
систему
{
    if ( this -> status == 1 ) // состояние
    готовности корневого объекта равно 1
    {
        return 0; // завершить работу метода
    }

    cout << "Object tree" << endl; // вывод заголовка
    this -> print_tree ( ); // отображение

```

иерархии

```
    cl_base * current_object = this;                // изначально корневой
    объект является текущим
    string command;                                // строка для хранения
    команды
    string param;                                  // строка для хранения
    параметра команды

    while ( true )                                // начало бесконечного цикла
    {
        cin >> command;                            // считать команду

        if ( command == "END" )                    // команда END
        {
            break;                                  // завершить чтение команд
        }

        cin >> param;                              // считывание параметра
    команды

        if ( command == "SET" )                    // команда SET
        {
            cl_base * found = current_object -> get_subobject_by_coordinate
            ( param );

            if ( found != nullptr )                 // объект найден
            {
                current_object = found;              // установить текущий
    объект
                cout << "Object is set: " << found -> get_name ( ) << endl;
            }
            else                                     // объект не найден
            {
                cout << "The object was not found at the specified coordinate: "
    << param << endl;
            }
        }
        else if ( command == "FIND" )               // команда FIND
        {
            cl_base * found = current_object -> get_subobject_by_coordinate
            ( param );

            if ( found != nullptr )                 // объект найден
            {
                cout << param << "    Object name: " << found -> get_name ( ) <<
    endl;
            }
            else                                     // объект не найден
            {
                cout << param << "    Object is not found" << endl;
            }
        }
        else if ( command == "MOVE" )               // команда MOVE
        {
            cl_base * new_head = current_object -> get_subobject_by_coordinate (
```

```

param );

        if ( new_head == nullptr )                // головной объект не
найден
        {
            cout << param << "      Head object is not found" << endl;
            continue;                               // переход к следующей итерации
        }

                                                    // проверка, что новый головной
объект не принадлежит ветке текущего
        cl_base * temp = new_head;                // временный указатель
        while ( temp != nullptr )                 // пока не достигнут корень
        {
            if ( temp == current_object )          //      новый      головной
принадлежит ветке текущего
            {
                cout << param << "      Redefining the head object failed" <<
endl;
                break;                             // прервать цикл
            }
            temp = temp -> get_head ( );           // переход к головному
объекту
        }

        if ( temp != nullptr )                    // если найден в ветке
        {
            continue;                             // переход к следующей команде
        }

        if ( new_head -> get_subobject_by_name ( current_object -> get_name
( ) ) != nullptr )
        {
            cout << param << "      Dubbing the names of subordinate objects"
<< endl;
        }
        else if ( current_object -> set_head ( new_head ) )
        {
            // переопределение удалось
            cout << "New head object: " << new_head -> get_name ( ) << endl;
        }
        else
        {
            // переопределение не удалось
            cout << param << "      Redefining the head object failed" <<
endl;
        }
    }
    else if ( command == "DELETE" )                // команда DELETE
    {
        cl_base * to_delete = current_object -> get_subobject_by_name
( param );

        if ( to_delete != nullptr )                // подчинённый объект
найден
        {
                                                    // построение абсолютного пути для

```

```

вывода
    string abs_path; // строка для пути
    cl_base * path_builder = to_delete; // временный указатель

    while ( path_builder != nullptr ) // пока не достигнут корень
    {
        if ( path_builder -> get_head ( ) != nullptr )
        {
            // добавить имя в начало
            abs_path = "/" + path_builder -> get_name() + abs_path;
        }
        path_builder = path_builder -> get_head ( ); // переход к
головному
    }

    current_object -> delete_subobject ( param ); // удалить
подчинённый объект
    cout << "The object " << abs_path << " has been deleted" << endl;
    }

// если объект не найден, ничего
не выводить
    }
}

    cout << "Current object hierarchy tree" << endl; // вывод заголовка
    this -> print_tree ( ); // отображение
итоговой иерархии

    return 0;
}

```

5.12 Файл cl_application.h

Листинг 12 – cl_application.h

```

#ifndef __CL_APPLICATION__H
#define __CL_APPLICATION__H

#include "cl_base.h"

class cl_application : public cl_base
{
public:
    cl_application ( cl_base * p_head_object, std :: string s_object_name =
"Application" ); // параметризованный конструктор
    void build_tree_objects ( );
// строит исходное дерево иерархии объектов: чтение входных данных, создание
объектов с проверкой уникальности имён и правил добавления
    int exes_app ( );
// запускает систему: чтение команд редактирования имён объектов по уровням
и индексам, изменение имён и вывод обновлённого дерева

```

```
};  
  
#endif
```

5.13 Файл cl_base.cpp

Листинг 13 – cl_base.cpp

```
#include "cl_base.h"  
#include <iostream>  
#include <sstream>  
using namespace std;  
  
cl_base :: cl_base ( cl_base * p_head_object, string s_object_name )    //  
параметризированный конструктор  
{  
    this -> s_object_name = s_object_name;  
    // хранение наименования объекта  
    this -> p_head_object = p_head_object;  
    // хранение адреса на головной объект  
  
    if ( p_head_object ) {  
        // есть ли головной объект?  
        p_head_object -> subordinate_objects.push_back ( this );    //  
добавляем адрес текущего объекта в список адресов подчинённых объектов  
головного объекта  
    }  
}  
  
cl_base :: ~cl_base ( )  
    // деструктор  
{  
    for ( int i = 0; i < this -> get_subordinate_count ( ); i++ )    //  
    {  
        значение индекса i меньше размера вектора подчинённых объектов?  
        delete subordinate_objects [ i ];  
        // удалить динамически созданный объект, адрес которого хранится в i-  
        том элементе массива подчинённых объектов текущего объекта  
    }  
}  
  
bool cl_base :: set_name ( string new_name )  
    // редактирует имя объекта с проверкой отсутствия дуближа среди  
подчинённых объектов у головного  
{  
    if ( this -> p_head_object != nullptr )  
        // есть ли головной объект?  
    {  
        for ( int i = 0; i < p_head_object -> get_subordinate_count ( ); i++ )  
        {  
            //
```

```

значение индекса i меньше количества подчинённых объектов головного объекта
    if ( p_head_object -> subordinate_objects [ i ] -> get_name ( ) ==
new_name )
    {
        проверяем уникальность имени
        return false;
    }
    есть другой подчинённый с таким же именем
}

this -> s_object_name = new_name;
// установка нового имени
return true;
// имя успешно изменено
}

string cl_base :: get_name ( )
// возвращает наименование объекта
{
    return this -> s_object_name;
// текущее наименование объекта
}

cl_base * cl_base :: get_head()
// возвращает адрес головного объекта текущего объекта
{
    return this -> p_head_object;
// адрес головного объекта текущего объекта
}

int cl_base :: get_subordinate_count ( )
// возвращает количество подчинённых объектов текущего объекта
{
    return (int) this -> subordinate_objects.size ( );
// текущее количество подчинённых объектов
}

cl_base * cl_base :: get_subobject_by_name ( string name )
// возвращает адрес подчинённого объекта по его имени (если не найден -
возвращает nullptr)
{
    for ( int i = 0; i < this -> get_subordinate_count ( ); i++ )
    {
        значение индекса i меньше количества элементов в списке подчинённых объектов
        текущего объекта
        {
            if ( subordinate_objects [ i ] -> s_object_name == name )
            {
                имя текущего подчинённого объекта совпадает со значением входного параметра
                name
                return subordinate_objects [ i ];
            }
            // вернуть значение i-того указателя из массива подчинённых объектов
        }
    }

    return nullptr;
}

```

```

        // объект не найден
    }

    cl_base * cl_base :: get_subobject_by_index ( int index )
    // возвращает адрес подчинённого объекта по его индексу (от 1 до
    количества подчинённых)
    {
        if ( index < 1 || index > this -> get_subordinate_count ( ) ) //
        проверка на корректность индекса
        {
            return nullptr;
            // объект не найден
        }

        return subordinate_objects [ index - 1 ];
        // такой подчинённый существует
    }

    cl_base * cl_base :: get_subobject_on_branch ( string name ) //
    метод поиска объекта на ветке дерева иерархии от текущего по имени
    {
        vector < cl_base * > queue;
        // массив для хранения очереди обхода
        queue.push_back ( this );
        // добавить адрес текущего объекта в очередь
        cl_base * p_found = nullptr;
        // инициировать указатель на объект базового класса p_found значением
        нулевого указателя

        for ( int i = 0; i < queue.size ( ); i++)
            // значения индекса i меньше количества элементов в очереди queue
        {
            if ( queue [ i ] -> get_name ( ) == name )
                // имя i-го элемента из queue совпадает с искомым
            {
                if ( p_found != nullptr )
                    // значение указателя p_found не нулевое
                {
                    return nullptr;
                    // вернуть значение нулевого указателя
                }
                p_found = queue [ i ];
                // присвоить переменной p_found значение i-го элемента из queue
            }

            for ( int j = 1; j <= queue [ i ] -> get_subordinate_count ( ); j++ )//
            значение индекса j меньше количества подчинённых объектов у i-го элемента из
            queue
            {
                queue.push_back ( queue [ i ] -> get_subobject_by_index ( j ) );//
                добавить в очередь адрес найденного элемента по индексу j по указателю на i-
                ый элемент массива queue
            }
        }

        return p_found;
    }

```

```

        // вернуть адрес найденного объекта p_found
    }

    cl_base * cl_base :: get_subobject_on_tree ( string name )
    // метод поиска объекта на всём дереве иерархии по имени
    {
        cl_base * root = this;
        // иниц. указатель root значением адреса текущего объекта

        while ( root -> get_head() != nullptr )
            // у объекта, на который указывает root, есть головной объект
            {
                root = root -> get_head ( );
                // присвоить указателю root адрес его головного объекта
            }

        return root -> get_subobject_on_branch ( name );
        // вернуть результат вызова метода get_url_on_branch по указателю root на
        // корневой объект root
    }

    void cl_base :: print_tree ( int depth )
    // метод вывода иерархии объектов (дерева или ветки) от текущего
    // объекта
    {
        for ( int i = 0; i < depth; i++ )
            // вывод отступа: по 4 пробела на каждый уровень вложенности
            {
                cout << "    ";
                // вывод на консоль 4 пробелов
            }

        cout << this -> get_name ( ) << endl;
        // вывод имени текущего объекта и переход на новую строку
        for ( int j = 0; j < this -> get_subordinate_count ( ); j++ )
            {
                subordinate_objects [ j ] -> print_tree ( depth + 1 );
                // вызов метода print_tree рекурсивно по указателю на i-тый элемент массива
                // подчинённых объектов с глубиной отступа +1 в качестве аргумента
            }
    }

    void cl_base :: print_ready ( int depth )
    // метод вывода иерархии объектов и состояний их готовности от текущего
    // объекта
    {
        for ( int i = 0; i < depth; i++ )
            // значение переменной i меньше значения входного параметра depth
            {
                cout << "    ";
                // вывод на консоль 4 пробелов
            }

        cout << this -> get_name ( );
        // вывод на консоль имени текущего объекта
    }

```

```

    if ( this -> status != 0 )
        // значение поля status текущего объекта не равно нулю
    {
        cout << " is ready" << endl;
        // объект готов
    }
    else
    {
        cout << " is not ready" << endl;
        // объект не готов
    }

    for ( int j = 0; j < this -> get_subordinate_count ( ); j++ )           //
        // значение индекса j меньше размера массива подчинённых объектов
    {
        subordinate_objects [ j ] -> print_ready ( depth + 1 );
        // вызов метода print_ready рекурсивно по указателю на i-тый элемент
        // массива подчинённых объектов с глубиной отступа +1 в качестве аргумента
    }
}

void cl_base :: set_ready ( int state )
{
    if ( state != 0 )
        // включение объекта
    {
        // проверка готовности для всех вышестоящих объектов до корня
        cl_base * p = this -> p_head_object;
        // начать с головного объекта, текущий объект не проверяется

        while ( p != nullptr )
            // пока не достигнут корень
        {
            if ( p -> status == 0 )
                // обнаружен вышестоящий объект в выключенном состоянии
            {
                return;
            }
            // установка готовности для текущего объекта невозможна
        }

        p = p -> get_head ( );
        // переход к головному объекту (подъём на уровне выше)
    }

    this -> status = state;
    // установка состояния готовности текущего объекта
}
else
    // выключение объекта (передано нулевое значение)
{
    this -> status = 0;
    // сброс состояния готовности текущего объекта

    for ( int i = 0; i < this -> get_subordinate_count ( ); i++ )           //

```

```

цикл по всем подчинённым объектам
{
    subordinate_objects [ i ] -> set_ready ( 0 );           //
    рекурсивный вызов метода set_ready для отключения подчинённых объектов
}
}

bool cl_base :: set_head ( cl_base * new_head )
    // метод переопределения головного объекта для текущего объекта
{
    if ( this -> p_head_object == nullptr || new_head == nullptr )           //
        // переопределение головного объекта для корневого объекта недопустимо и нельзя
        // создать второй корневой объект
        {
            return false;
        }

    cl_base * ancestor = new_head;
    // инициировать указатель ancestor адресом нового головного

    while ( ancestor != nullptr )
        // пока не достигнут корень
        {
            if ( ancestor == this )
                // новый головной принадлежит ветке текущего
                {
                    return false;                                           //
                }
            вернуть ложь
            ancestor = ancestor -> get_head ( );
            // переход к головному объекту
        }

    if ( new_head -> get_subobject_by_name ( this -> get_name ( ) ) !=
        nullptr )
        {
            // у
            // нового головного уже есть подчинённый с таким же именем
            return false;
            // дуближ имени у головного, вернуть ложь
        }

    for ( int i = 0; i < this -> p_head_object -> get_subordinate_count ( );
        i++ )
        {
            //
            // удаление адреса текущего объекта из списка адресов подчинённых объектов
            // старого головного объекта
            if ( this -> p_head_object -> subordinate_objects [ i ] == this )
                {
                    //
                    найден текущий объект в списке старого головного
                    this -> p_head_object -> subordinate_objects.erase ( this ->
                    p_head_object -> subordinate_objects.begin ( ) + i );
                    break;
                    // выход из цикла
                }
        }
}

```

```

    }

    this -> p_head_object = new_head;
    // установить новый головной объект
    new_head -> subordinate_objects.push_back ( this );
    // добавить адрес текущего объекта в список адресов подчинённых объектов
    // нового головного объекта
    return true;
    // вернуть истину
}

void cl_base :: delete_subobject ( string name )
    // метод удаления подчинённого объекта по наименованию
{
    for ( int i = 0; i < this -> get_subordinate_count ( ); i++ )
    {
        if ( subordinate_objects [ i ] -> get_name ( ) == name )
            {
                // найден подчинённый объект с указанным именем
                delete subordinate_objects [ i ];
                // удалить объект
                subordinate_objects.erase ( subordinate_objects.begin ( ) + i );
                return;
            }
        // завершить метод
    }
    // если объект не найден, метод завершает работу
}

cl_base * cl_base :: get_subobject_by_coordinate ( string coordinate ) //
    // метод получения объекта по координате
{
    if ( coordinate.empty ( ) )
        // координата - пустая строка
        {
            return nullptr;
            // вернуть значение нулевого указателя
        }

    if ( coordinate == "." )
        // текущий объект: координата равна "."
        {
            return this;
            // вернуть адрес текущего объекта
        }

    cl_base * root = this;
    // поиск корневого объекта. инициировать указатель root адресом текущего
    // объекта
    while ( root -> get_head ( ) != nullptr )
        // пока есть головной объект
        {
            root = root -> get_head ( );
            // подняться на уровень вверх
        }
}

```

```

    if ( coordinate == "/" )
    // корневой объект: координата равна "/"
    {
        return root;
    // вернуть адрес корневого объекта
    }

    if ( coordinate.length ( ) >= 2 && coordinate[0] == '/' && coordinate[1]
== '/' )
    {
        //
        поиск по уникальному имени от корня //имя
        return root -> get_subobject_on_tree ( coordinate.substr ( 2 ) );//
        вернуть результат поиска по дереву
    }

    if ( coordinate[0] == '.' )
        // поиск по уникальному имени от текущего .имя
    {
        return this -> get_subobject_on_branch ( coordinate.substr ( 1 ) );//
        вернуть результат поиска на ветке от текущего
    }

    //
    обработка абсолютного /путь/к/объекту или относительного путь/к/объекту
    stringstream ss ( coordinate );
    // создание потока для разбора строки
    string part;
    // хранение части пути
    cl_base * cur;
    // текущий объект при обходе

    if ( coordinate[0] == '/' )
        // абсолютный путь
    {
        ss.get ( );
        // пропуск первого слеша
        cur = root;
        // начинаем от корня
    }
    else
        //
    относительный путь
    {
        cur = this;
        // начинаем от текущего
    }

    while ( getline ( ss, part, '/' ) )
        // разделение строки по слешу
    {
        if ( part.empty ( ) )
            // пропуск пустых частей
        {
            break;
        }
    }

```

```

        cl_base * found = cur -> get_subobject_by_name ( part );           //
поиск подчинённого с таким именем

        if ( found == nullptr )
            // объект не найден
            {
                return nullptr;
            }
        // вернуть значение нулевого указателя

        cur = found;
        // переход к найденному объекту
    }

    return cur;
    // вернуть адрес найденного объекта
}

```

5.14 Файл cl_base.h

Листинг 14 – cl_base.h

```

#ifndef __CL_BASE__H
#define __CL_BASE__H

#include <string>
#include <vector>
using namespace std;

class cl_base
{
private:
    string s_object_name;
        // хранение наименования объекта
    cl_base * p_head_object;
        // хранение адреса на головной объект
    vector < cl_base * > subordinate_objects;
        // динамический массив адресов подчинённых объектов

protected:
    int status;
        // состояние готовности объекта

public:
    cl_base ( cl_base* p_head_object, string s_object_name = "Base_object" );
        // параметризованный конструктор
    ~cl_base ( );
        // деструктор для удаления текущего объекта и всех его подчинённых
    объектов
}

```

```

    bool set_name ( string new_name );
        // редактирует имя объекта с проверкой отсутствия дуближа среди
    подчинённых объектов у головного
    string get_name ( );
        // возвращает наименование объекта
    cl_base * get_head ( );
        // возвращает адрес головного объекта текущего объекта
    int get_subordinate_count ( );
        // возвращает количество подчинённых объектов текущего объекта
    cl_base * get_subobject_by_name ( string name );
        // возвращает адрес подчинённого объекта по его имени (если не
    найден - возвращает nullptr)
    cl_base * get_subobject_by_index ( int index );
        // возвращает адрес подчинённого объекта по его индексу (от 1 до
    количества подчинённых)

    cl_base * get_subobject_on_branch ( string name );
        // метод поиска объекта на ветке дерева иерархии от текущего по
    имени
    cl_base * get_subobject_on_tree ( string name );
        // метод поиска объекта на всём дереве иерархии по имени
    void print_tree ( int depth = 0 );
        // метод вывода иерархии объектов (дерева или ветки) от текущего
    объекта
    void print_ready ( int depth = 0 );
        // метод вывода иерархии объектов и состояний их готовности от
    текущего объекта
    void set_ready ( int state );
        // метод установки состояния готовности для текущего объекта

    bool set_head ( cl_base * new_head );
        // метод переопределения головного объекта для текущего объекта
    void delete_subobject ( string name );
        // метод удаления подчинённого объекта по наименованию
    cl_base * get_subobject_by_coordinate ( string coordinate );
        // метод получения объекта по координате
};

#endif

```

5.15 Файл main.cpp

Листинг 15 – main.cpp

```

#include <stdlib.h>
#include <stdio.h>

#include "cl_application.h"
using namespace std;

```

```
int main()
{
    cl_application ob_cl_application ( nullptr ); // создание корневого
объекта
    ob_cl_application.build_tree_objects ( );      // конструирование системы,
построение дерева объектов
    return ob_cl_application.exec_app ( );        // запуск системы
}
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 7.

Таблица 7 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
<pre> rootela / object_1 3 / object_2 2 /object_2 object_4 3 /object_2 object_5 4 / object_3 3 /object_2 object_3 6 /object_1 object_7 5 /object_2/object_4 object_7 3 endtree FIND object_2/object_4 SET /object_2 FIND //object_7 FIND object_4/object_7 FIND . FIND .object_7 FIND object_4/object_7 MOVE .object_7 SET object_4/object_7 MOVE //object_1 MOVE /object_3 END </pre>	<pre> Object tree rootela object_1 object_7 object_2 object_4 object_7 object_5 object_3 object_3 object_2/object_4 Object name: object_4 Object is set: object_2 //object_7 Object is not found object_4/object_7 Object name: object_7 . Object name: object_2 .object_7 Object name: object_7 object_4/object_7 Object name: object_7 .object_7 Redefining the head object failed Object is set: object_7 //object_1 Dubbing the names of subordinate objects New head object: object_3 Current object hierarchy tree rootela object_1 object_7 object_2 object_4 object_5 </pre>	<pre> Object tree rootela object_1 object_7 object_2 object_4 object_7 object_5 object_3 object_3 object_2/object_4 Object name: object_4 Object is set: object_2 //object_7 Object is not found object_4/object_7 Object name: object_7 . Object name: object_2 .object_7 Object name: object_7 object_4/object_7 Object name: object_7 .object_7 Redefining the head object failed Object is set: object_7 //object_1 Dubbing the names of subordinate objects New head object: object_3 Current object hierarchy tree rootela object_1 object_7 object_2 object_4 object_5 </pre>

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
	<pre> object_3 object_3 object_7 </pre>	<pre> object_3 object_3 object_7 </pre>
<pre> Root / object_1 2 / object_1 3 / object_2 7 /object_3 object_4 3 endtree END </pre>	<pre> / Dubbing the names of subordinate objects Object tree Root object_1 The head object /object_3 is not found </pre>	<pre> / Dubbing the names of subordinate objects Object tree Root object_1 The head object /object_3 is not found </pre>
<pre> Root / A 2 /A B 3 /A C 4 /A/B D 5 endtree FIND / FIND . FIND //C FIND .B FIND A/B/D FIND /A/B FIND //X SET /A FIND . DELETE C SET /A/B MOVE /A SET /A/B MOVE /A/B/D END </pre>	<pre> Object tree Root A B D C / Object name: Root . Object name: Root //C Object name: C .B Object name: B A/B/D Object name: D /A/B Object name: B //X Object is not found Object is set: A . Object name: A The object /A/C has been deleted Object is set: B /A Dubbing the names of subordinate objects Object is set: B /A/B/D Redefining the head object failed Current object hierarchy tree Root A B D </pre>	<pre> Object tree Root A B D C / Object name: Root . Object name: Root //C Object name: C .B Object name: B A/B/D Object name: D /A/B Object name: B //X Object is not found Object is set: A . Object name: A The object /A/C has been deleted Object is set: B /A Dubbing the names of subordinate objects Object is set: B /A/B/D Redefining the head object failed Current object hierarchy tree Root A B D </pre>

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
Root / A 2 endtree SET /B MOVE /C END	Object tree Root A The object was not found at the specified coordinate: /B /C Head object is not found Current object hierarchy tree Root A	Object tree Root A The object was not found at the specified coordinate: /B /C Head object is not found Current object hierarchy tree Root A

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).